FairCom DB - dbnotify Service Secure Communications to MQ Broker with TLS/SSL/x509

The 'dbnotify' feature is an optional Plug-in service that is included in the FairCom DB package. This feature allows you to define "event-criteria" on any FairCom DB database file. Using these criteria, the file is actively monitored and a trigger will invoke a 'dbnotify' processing event if/when the criteria are satisfied. Reflecting this database event, a configurable MQTT message is sent to a specified MQTT Broker.

This document describes the steps necessary to secure the 'dbnotify' communications between the FairCom DB Server and the FairCom MQ Broker. OpenSSL and x509 authentication are implemented.

When you enable the 'dbnotify' service on your FairCom DB Server, the 'dbnotify' service technology acts as a TCP/IP Client into the FairCom MQ Broker. When a 'dbnotify' event is triggered in your DB Server, a database record is added into the MQTT topic's integration table on the Broker. This record represents an MQTT message, now securely stored under the broker.

Our security objective in this document is to secure the TCP/IP communication between the DB Server's 'dbnotify' service and the FairCom MQ broker. We will do this by activating TLS/SSL to encrypt the network communications. We will also implement x509 authorization to secure the user and password used for this connection. TLS is an updated, more secure version of SSL. We will use the term SSL throughout this document, yet you can assume this is synonymous with TLS.

x509 authentication extends the SSL support to allow database user authentication based on the client providing a certificate that is trusted by the server and having a user that is registered in the Server or Broker's authorized user's database (typically added via 'ctadmn' utility).

Certificates

In order to support SSL communications and x509 authorization we need a few sets of certificates and keys:

- 'ctree_ssl.pem'/'ctree_ssl.key' - FairCom MQ Broker (server-side) certificate file: This is a container that includes: public certificate and public key, private key, and root certificates. Note that 'ctree_ssl.pem' appends the ‘ctree_ssl.key’ and the rootca.pem to make this a

complete chain. 'ctree_ssl.pem' should be private to the server/broker because it contains this private key.

- 'ctsrvr.pem' - Client-side certificate file: The file will be used by the 'dbnotify' services to secure the TCP/IP communication. This is 'ctree_ssl.pem' without the ‘ctree_ssl.key’. It is used by a client to validate the server. In this exercise, the client is our 'dbnotify' service.

- 'rootca.pem'/'rootca.key' - Certificate Authority(CA): This certificate and key file is used to sign both client and server certificates. The ‘rootca.key’ should be highly protected, as this is the base "secret" on which all the security is based.

- 'admin.pem'/admin.key' - FairCom ADMIN user certificate and private key: Each unique database user, including FairCom's ADMIN user, will need their own certificate and private key, signed by a trusted Certificate Authority(CA). This one is intended for the "admin" account. Note that 'admin.pem' has the 'rootca.pem' appended to make it a complete "chain" for authentication.

- 'JonDoe.pem'/JonDoe.key' - Normal (non-admin) user certificate and private key: Each unique database user will need their own certificate and private key, signed by a trusted Certificate Authority(CA). This one is an example of a "JonDoe" account. Later, you will modify our script to create your own user certificates.

We have provided a working (createCerts.bat) script which will first create the Certificate Authority(CA), and then use this certificate to sign the other certificates. Edit this file to see the openssl '-subj' switch where later you should substitute your company's information. Around line 75, you will see our 'JonDoe' user. Keep this intact to test this demo, then later go back and adjust as you see fit for your company and users.

You will find that we have provided sample certificates for use with this demo in addition to a 'createCerts.bat' script in this folder:
\FairCom-DB.windows.64bit.vXX.X.X.XX\config\x509.auth.demo

Feel free to use our provided sample certificates, or use the 'createCerts.bat' script to create your own. The createCerts.bat script only needs the 'rootca.conf' configuration file. If you desire to create your own certificates, copy these two files to any working directory you choose. If you try to run createCerts.bat in the x509.auth.demo folder, you will get error messages conflicting with our existing certificate files and keys.

The “CreateCerts.bat” file also requires the OpenSSL.exe utility. Please be sure the utility is accessible from your command prompt before running createCerts.bat.

If you desire, execute the 'createCerts.bat' script from your Windows command prompt, Note that you will be prompted for a 'PEM pass phrase'. We used 'FairComQA', yet you may choose your own phrase.

> createCerts.bat

You will see:

> create root CA cert request and private key
> Enter PEM pass phrase: FairComQA

Initially, you are setting the 'pass phrase' for the Certificate Authority(CA) certificate, then you are also prompted when this certificate is being used to sign the other certificates.

OK, we have just created the certificate files (.pem) and keys (.key) we want.

ctree_ssl.pem
ctsrvr.pem
admin.key
admin.pem
JonDoe.key
JonDoe.pem

For your convenience, as mentioned above, our demo set of certificates and keys have also been placed into the 'x509.auth.demo' folder:
\FairCom-DB.windows.64bit.vXX.X.X.XX\config\x509.auth.demo

Test SSL on FairCom MQ Broker

Let's begin by testing our basic SSL using our MQ Broker.

Be sure you have shutdown both your DB and MQ Server as we make the following configuration changes.

1. FairCom MQ Broker (server-side) certificate: Copy the 'ctree_ssl.pem' file you generated above into your Broker's 'server folder:

   > copy .\YourWorkFolder\ctree_ssl.pem .\FairCom-MQ.win64.vX.x.x.xxx\server\ctree_ssl.pem

   FYI: We do not need the 'ctree_ssl.key' file for it is already embedded in 'ctree_ssl.pem'.

2. FairCom MQ Broker: Activate SSL: Edit your MQ Broker's 'ctsrvr.cfg file: .\config\ctsrvr.cfg

   If you are testing your Client on the same machine as your server, you will want to deactivate the Shared Memory Protocol. Even if you are connecting with TCP/IP, internally, if the Shared Memory protocol is activated, we will automatically switch the Shared Memory when possible. Therefore to test your TCP/IP TLS/SSL it is best to deactivate Shared Memory by commenting out its keyword in the ctsrvr.cfg file.

   ; Communication protocols
   ; COMM_PROTOCOL FSHAREMM

3. Un-comment the SSL support. Let's start by uncommenting just these four lines. We will look at the x509 later.

   ;Here is where you can activate (un-comment) SSL
   SUBSYSTEM COMM_PROTOCOL SSL {

   ;This is the file name in your server's directory
   SERVER_CERTIFICATE_FILE ctree_ssl.pem

   ;For SSL you can specify (un-comment) a debug log file name
   DEBUG_LOG ssl.log

   ;Here you can restrict access to SSL ONLY.
   SSL_CONNECTIONS_ONLY YES

4. Start your Broker: Execute '.\server\faircom.exe'

5. Now copy the 'ctsrvr.pem' file into your client Apps folder. In this exercise, we will use our ./tools folder in our Broker's package.

   > copy .\YourWorkFolder\ctsrvr.pem .\FairCom-MQ.win64.vX.x.x.xxx\tools\ctsrvr.pem

6. Here we will run a few command line Utils from our 'tools' folder to verify the SSL is enabled. Note that in the server's configuration file (ctsrvr.cfg) , we set that only SSL (SSL_CONNECTIONS_ONLY YES) connections are allowed:

   To request an SSL-enabled connection for an ISAM connection, append ^fssltcp to the server name. For example:

   ctixmg ADMIN ADMIN "FCEDGEMQ@localhost^fssltcp"

   ctadmn ADMIN ADMIN "" "FCEDGEMQ@localhost^fssltcp"

   Be sure to use quotes on the command line.

   If you get:
   'Could not initialize c-tree Plus(R) (1104)'
   most likely you did not copy 'ctsrvr.pem' into your Apps folder.

   To request an SSL-enabled connection for SQL, prepend ssl: to the connection string. For example:

   isql -u admin -p ADMIN ssl:6598@localhost:ctreesql

   ctadmn and c-treeACE Monitor show the communication protocol that is in use, including whether or not the connection is using SSL:

F_TCPIP indicates an unencrypted ISAM TCP/IP connection.
FSSLTCP indicates an SSL-enabled ISAM TCP/IP connection.
SQL_TCPIP indicates an unencrypted SQL TCP/IP connection.
SQL_SSLTCP indicates an SSL-enabled ISAM TCP/IP connection.

Example: run ctadmn:
> ctadmn ADMIN ADMIN "" "FCEDGEMQ@localhost^fssltcp"

**** FairCom(R) Server Administration Utility ****

1. User Operations
2. Group Definitions
3. File Security
4. Monitor Clients

Enter your choice (1-10), or 'q' to quit>> 4

Select 4 to "Monitor Clients":

Monitor Clients:

1. List Attached Clients
2. Kill Client

Enter your choice (1-2), or 'q' to return to previous menu>> 1

Select 1 to "List Attached Clients":

You might have to hit 'enter' a few times, for our 'ctixmg' might be last on the
List after other internal MQ threads.

UserID: ADMIN NodeName:
Task 19 Communications: FSSLTCP <<<<< SEE SSL
Memory: 163K Open Files: 3 Logon Time: 2:51
Tran Time: -- Rqst Time: 2:49 NoRequest Rqst# 196 OPNRFIL

Hit “q” to return to ctadmn’s main menu.

7. Add 'Jon Doe' to the FairCom MQ Broker's user database: While you are in ctadmn, go ahead and add the 'Jon Doe' user into your Server/Broker's database:

**** FairCom(R) Server Administration Utility ****

1. User Operations

Select option 1, “User Operations”.

User Operations:

1. Add New User
2. Remove Existing User
3. List Users

Enter your choice (1-9), or 'q' to return to previous menu>> 1
Select option 1, “Add New User”.

Enter User Id >> Jon Doe
Enter User Description >> Jon Doe

Answer the remaining prompts as you prefer. After the last prompt, press “q” to return to the main menu.

x509 Authorization: Activate

OK, now that we are assured our SSL communications channel is secure, let's activate x509 authorization.

8. Shut Down your FairCom MQ Broker: We are going to adjust the configuration file. Our updated setting will go into effect later when we re-start the Server/Broker. One quick way to stop the FairCom MQ Broker if you haven’t exited ctadmn.exe is choose option 7 “Stop Server” to shutdown the Broker.

9. FairCom MQ Broker: Activate x509 - Edit your MQ Broker's 'ctsrvr.cfg file: .\config\ctsrvr.cfg

   Un-comment the x509 options as shown:

```
;Require clients to provide a x509 certificate
VERIFY_CLIENT_CERTIFICATE YES

;Use x509 client certificate for database authentication
x509_AUTHENTICATION YES

;Use the SUBJECT:CN from the client's certificate as their user name
x509_PATH CN
```

}

10. Re-Start your FairCom MQ Broker: Our x509 is now activated on our Server/Broker.

11. Now copy the user certificate and key files you created above. Remember, we made an 'admin' and a 'JonDoe' user. Copy these files into the ./tools folder. Again, in this exercise, we will use our ./tools folder in our Broker's package.

    > copy .\YourWorkFolder\admin.pem .\FairCom-MQ.win64.vX.x.x.xxx\tools\admin.pem
    > copy .\YourWorkFolder\admin.key .\FairCom-MQ.win64.vX.x.x.xxx\tools\admin.key
    > copy .\YourWorkFolder\JonDoe.pem .\FairCom-MQ.win64.vX.x.x.xxx\tools\JonDoe.pem
    > copy .\YourWorkFolder\JonDoe.key .\FairCom-MQ.win64.vX.x.x.xxx\tools\JonDoe.key

12. Connect an ADMIN user with x509 authentication: Once your user's certificate (.pem) and key (.key) are in place on your client-side, you need to set two environment variables to specify which user. Here we set up the 'admin' user.

    set CTSSL_CLIENT_CERTIFICATE=admin.pem
    set CTSSL_CLIENT_KEY=admin.key

    Run your apps, without a user or password. They should run fine, now using the x509 certificate to supply the necessary user and password credentials.

    ctixmg none none "FCEDGEMQ@localhost^fssltcp"

    ctadmn none none "" "FCEDGEMQ@localhost^fssltcp"

13. Connect a specific user (JonDoe) with x509 authentication: Once your user's certificate (.pem) and key (.key) are in place on your client-side, you need to set two environment variables to specify which user. Here we set up the 'JonDoe' user.

    set CTSSL_CLIENT_CERTIFICATE=JonDoe.pem
    set CTSSL_CLIENT_KEY=JonDoe.key

    If you added the 'Jon Doe' user above, then you will be OK. If you did not add the 'Jon Doe' user yet, and if you try now, you should get an error.

    Run your apps, without a user or password. They should run fine now, using the x509 certificate to supply the necessary user and password credentials.

    ctixmg none none "FCEDGEMQ@localhost^fssltcp"

    If you are having trouble, use this client-side environment variable to activate SSL logging:

    set CTSSL_DEBUG_LOG=ssldebug.log

    Look for the arbitrary file name you set (ssldebug.log in our example) to be created where you are executing your client application.

    Possible Errors:
    Could not initialize c-tree Plus(R) (1104) -
    most likely you did not copy 'ctsrvr.pem' into your Apps folder.
    or
    Could not logon to server 'FCEDGEMQ@localhost^fssltcp'. Error #162.
    Server has broken connection with application.

    Typically this is because you either do not have your user certificate (JonDoe.pem) and key (JonDoe.key) files in place or you have not set your Environment variables.

    set CTSSL_CLIENT_CERTIFICATE=JonDoe.pem
    set CTSSL_CLIENT_KEY=JonDoe.key

Implement x509 authentication for the DB 'dbnotify' service

OK, we finally reached the point for our overall objective: secure the 'dbnotify' connection to the MQ Broker with x509 authentication.

Note: In our first exercise, we ensured the 'dbnotify' was working properly. Then we tested both basic SSL and x509 connections against our MQ Broker. Now we will turn on SSL and x509 support for 'dbnotify'. Let's focus first on the DB Server.

14. Shutdown your FairCom DB Server.

15. Edit the 'dbnotifyconnections.json' file in your DB 'config' folder. Enable the 'tls' and the 'x509_authentication' as shown below.

    See the 'dbnotifyconnections.json' file in your DB 'config' folder.

```
DB: See: .\config\dbnotifyconnections.json"
{
  "directConnectionsToFairComMq":
  [
    {
      "brokerConnectionName": "brokerCtree",
      "faircomServerName": "FCEDGEMQ",
      "brokerHostname": "localhost",
      "brokerUserName": "ADMIN",
      "brokerUserPassword": "ADMIN",
      "passwordCertificateFilename": "",
      "reconnectFrequencySeconds": 15,
      "tls":
      {
        "certificateFilename": "ctsrvr.pem",
        "enabled": true,                          <<<<< HERE <<<<<

        "x509_authentication":
        {
          "certificateFilename": "admin.pem",
          "privateKeyFilename": "admin.key",
          "enabled": true                         <<<<< HERE <<<<<
        }
      },
      "metadata": {},
      "defaultIncludePrimaryKey": "never",
      "defaultTagChanges":       "neverTag",
    }
  ]
}
```

16. Next, we need to copy the 'ctsrvr.pem' file into your FairCom DB server folder. The 'dbnotify' acts as a client into the MQ Broker, so we need the client-side certificate in place for the dbnotify.dll

    > copy .\YourWorkFolder\ctsrvr.pem .\FairCom-MQ.win64.vX.x.x.xxx\server\ctsrvr.pem

    We also need to copy your 'admin' user certificate and key files into the FairCom DB's server folder:

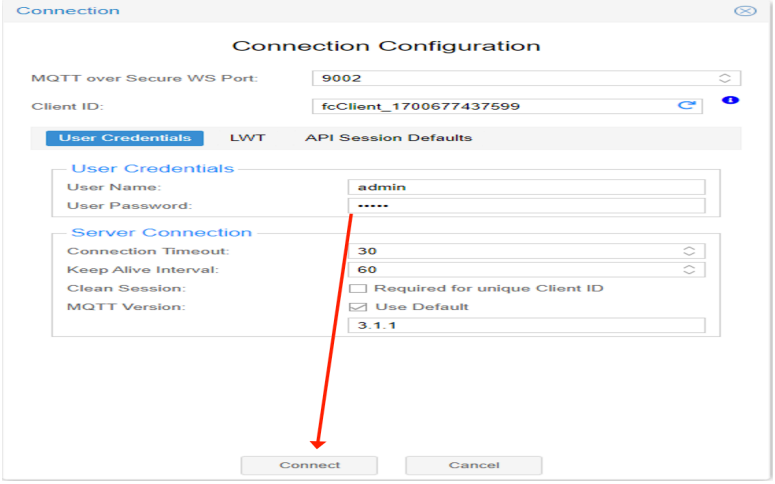
    > copy .\YourWorkFolder\admin.pem .\FairCom-MQ.win64.vX.x.x.xxx\server\admin.pem
    > copy .\YourWorkFolder\admin.key .\FairCom-MQ.win64.vX.x.x.xxx\server\admin.key

17. Now re-start your FairCom DB Server.

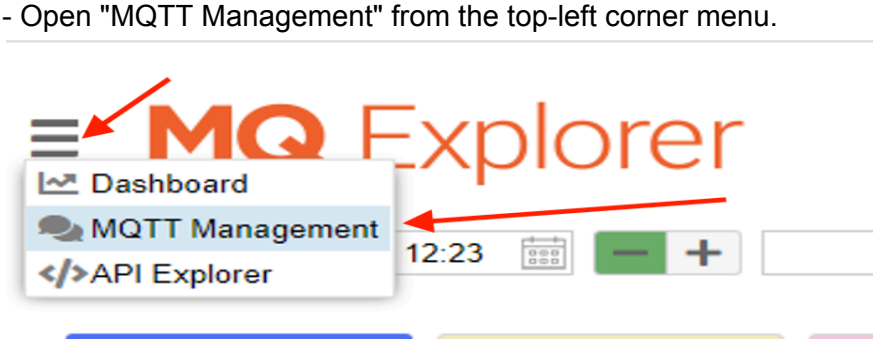
18. MQExplorer: Subscribe to "topic1" in the MQExplorer: Use your MQExplorer to see your messages not running with secure SSL x509 protection. From your Browser, load MQExplorer:

    https://localhost:8444/MQExplorer

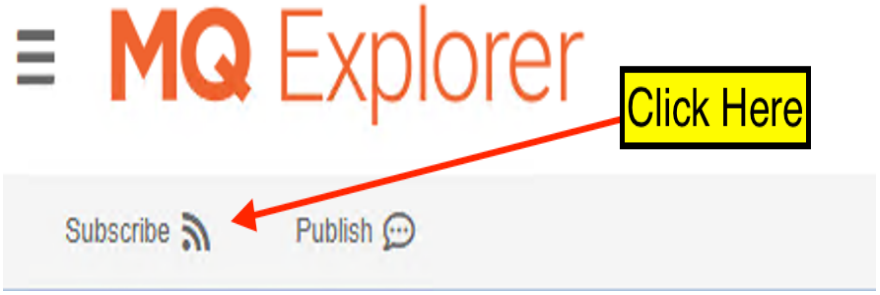
    Connect with the default credentials.



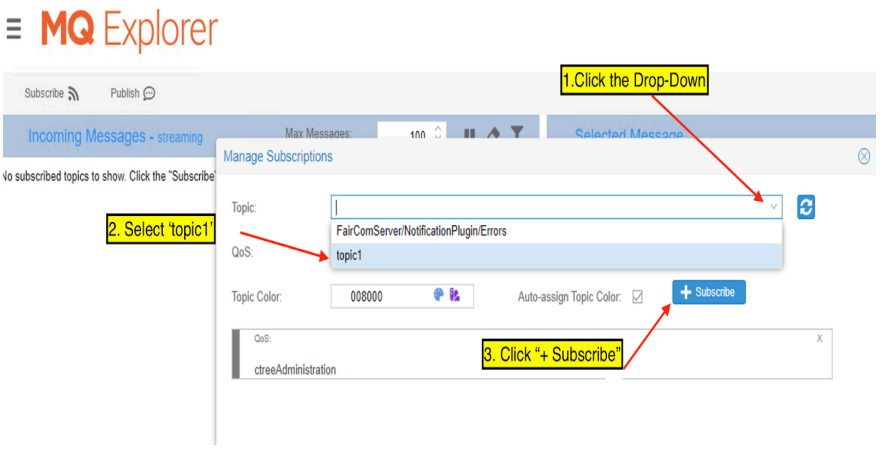
- Open "MQTT Management" from the top-left corner menu.



- Click at "Subscribe".



Select "topic1". Click at "+ Subscribe" and close this window.



Hit Close.

Leave this window open to see "Incoming Messages".

19. Instigate some 'dbnotify' activity and view MQTT messages in MQExplorer:

    DB: Return to your FairCom DB './tools' folder and execute 'custmast2.sql'. This will trigger a 'dbnotify' event, and then you can inspect your MQTT Broker for messages sent to 'topic1'.

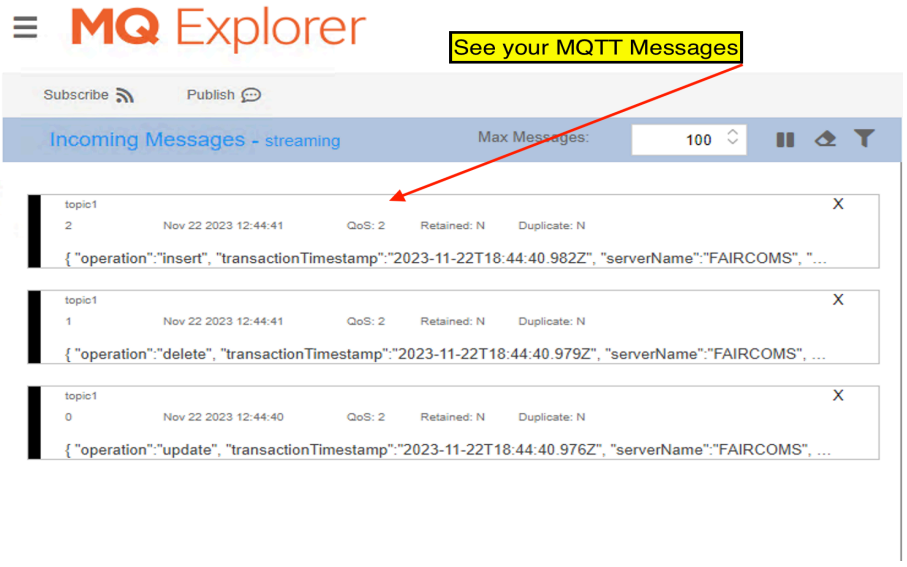
    From your DB ./tools folder, execute

    isql -s custmast2.sql -u ADMIN -a ADMIN ctreeSQL

    This script executes an update, a delete and an insert command for 'custmast' that will instigate notifications.

    Return to your MQExplorer to see the messages. It is fine to run 'custmast2.sql' over

and over again to see more incoming messages into MQExplorer.



OK, we are good. Activity from DB Server is now reflected in the MQ Broker via MQTT messages.

x509 Authentication MQTT Topic prerequisite

In our previous exercise, when we initially activated 'dbnotify' for the FairCom DB Server, we had not activated x509. When x509 is not activated, the topic named "topic1' on the FairCom MQ Broker is automatically created.

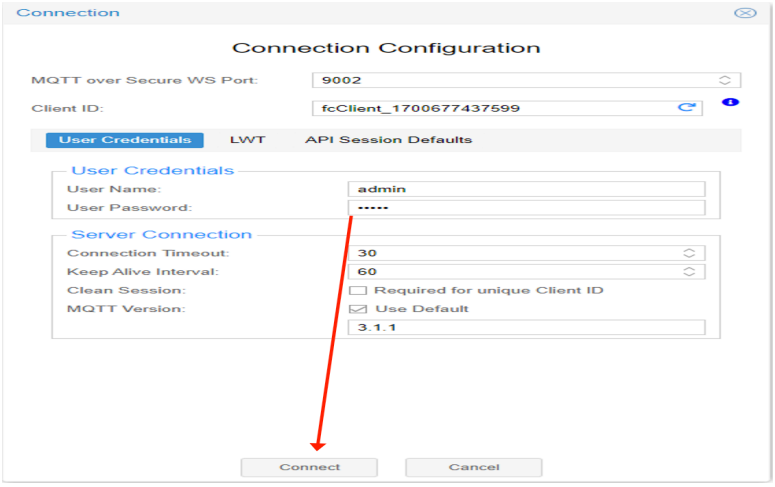
When ‘dbnotify’ x509 is activated, topics must be created manually on the FairCom MQ Broker. This is done with the MQExplorer using a few JSON scripts. We expect you to modify these JSON scripts to create your own topics.

These steps are not necessary now, for we have completed our demonstration. However if x509 is active, you will need to do the following to manually create new topics.

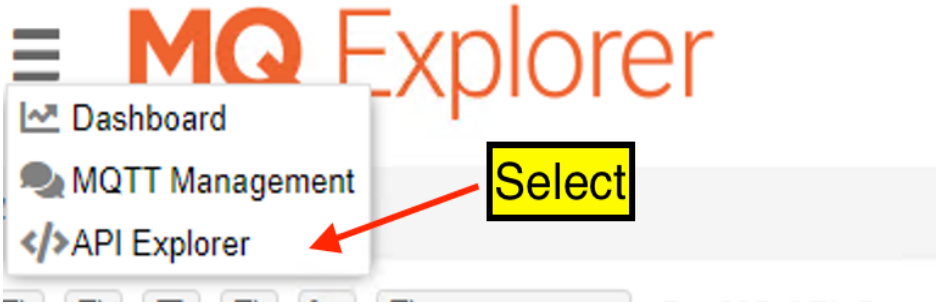
20. Load MQExplorer: https://localhost:8444/MQExplorer

    https://localhost:8444/MQExplorer

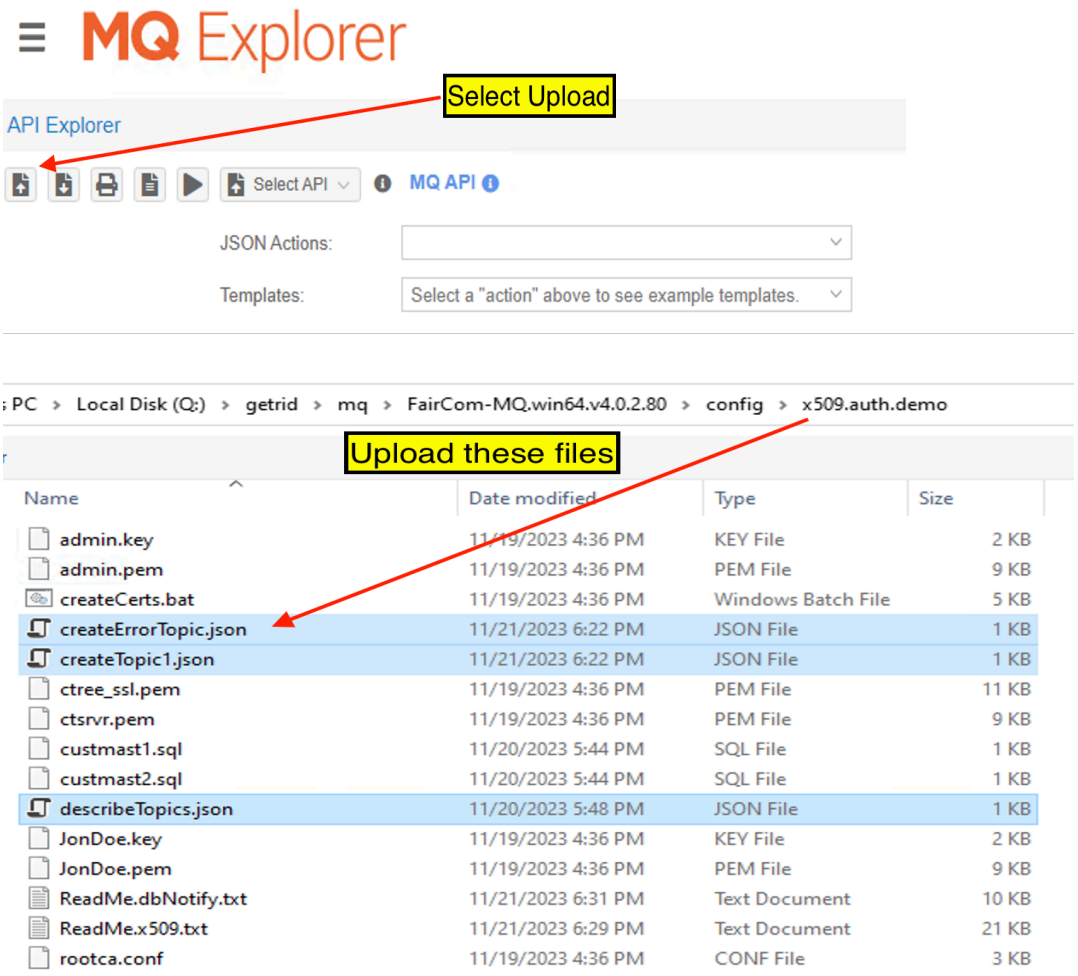
    Connect with the default credentials.



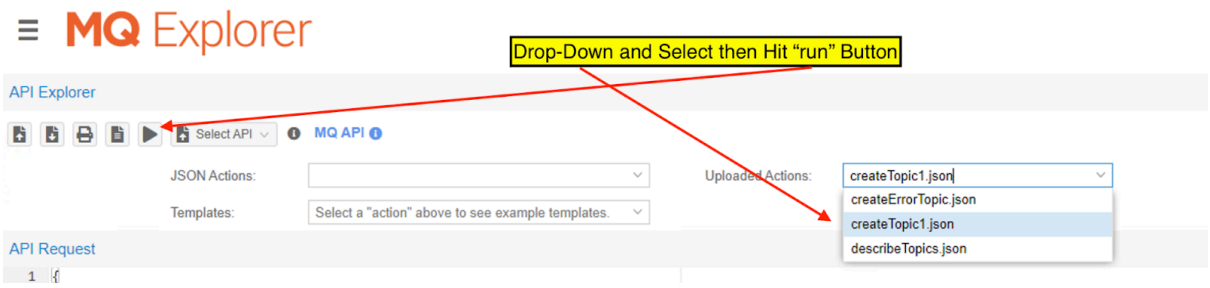
    Open "API Explorer" from the top-left corner menu.



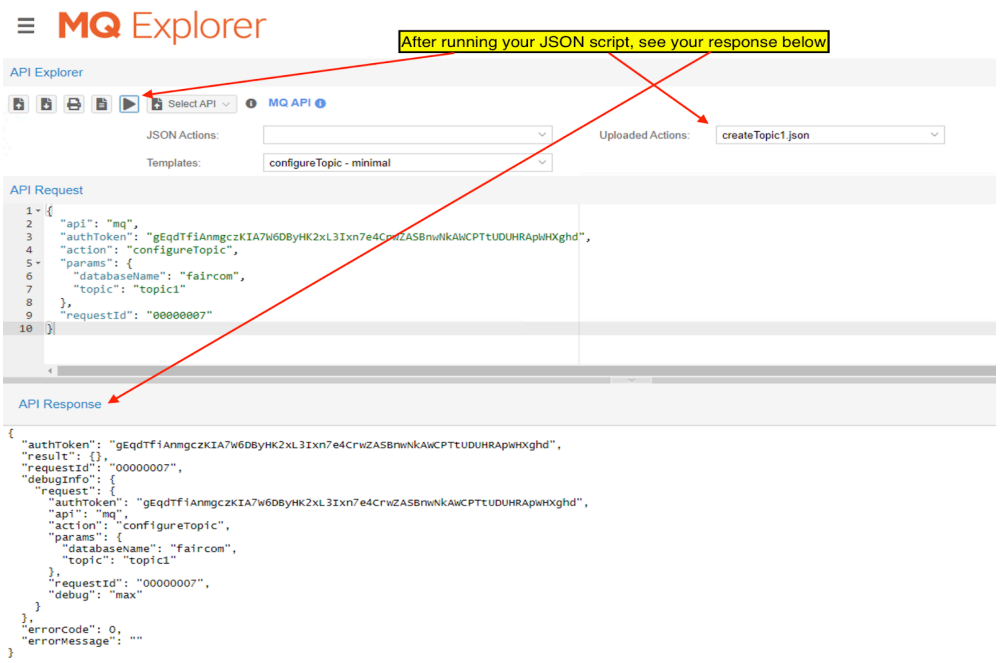
    Click at "Upload set of JSON files". Select:
    createErrorTopic.json;
    createTopic1.json;
    describeTopics.json



From the Uploaded Actions drop down, select each script and Click the "run" button.



See the results:



21. Debugging. If you have trouble resolving any errors, you can enable a debug log on both the server side and client side.

    Notice one of the keywords you uncommented in step 2 in the Broker ctsrvr.cfg was: DEBUG_LOG ssl.log

    You can review the server side ssl.log (typically located in the directory where faircom.exe is executed from) for server-side details.

    You can also enable client-side debugging by setting the following environment variable:

    CTSSL_DEBUG_LOG=YourFile

    Where “YourFile” is an arbitrary file name. Look for this file to be created where you are executing your client application.

Note: We intend ‘dbnotify’ to support automatic creation of topics with x509 in the future.

END OF NOTES